32. Worker Pools, Fan-in/out, Backpressure

32. Worker Pools, Fan-in/out, Backpressure

Phase: Phase 5 — Concurrency
Estimated time: 2 days
Slug: `32-worker-pools`

Learning objectives

- Build a bounded worker pool
- Apply fan-out / fan-in patterns
- Implement backpressure with buffered channels
- Bound concurrency in servers

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Worker pools bound concurrency. Fan-in/out and backpressure keep systems stable under load instead of spawning unbounded goroutines.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Fixed workers pull jobs from a channel.
- Buffered channels provide limited queueing (backpressure).
- Fan-out: many workers; fan-in: merge results.
- Close job channels to signal completion.
- Context cancels in-flight work when the caller leaves.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-worker && cd /tmp/gw-worker
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "sync"
)

func main() {
    jobs := make(chan int, 5)
    results := make(chan int, 5)
    var wg sync.WaitGroup
    for w := 0; w < 3; w++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for j := range jobs {
                results <- j * 2
            }
        }()
    }
    for j := 1; j <= 5; j++ {
        jobs <- j
    }
    close(jobs)
    wg.Wait()
    close(results)
    for r := range results {
        fmt.Println(r)
    }
}
```

Run it:

```
go run .
```

Expected output:

```
2
4
6
8
10
(order may vary)
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Build a 3-worker pool doubling ints.
2. Lower buffer to 0 and observe backpressure.
3. Add context cancel mid-flight.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Worker pool

```
jobs := make(chan Job)
results := make(chan Result)
for i := 0; i < workers; i++ {
    go worker(ctx, jobs, results)
}
```

Bound the number of goroutines doing expensive work. Unbounded `go` per request is a common outage cause.

Backpressure

When consumers are slow, producers must block or drop according to policy. Buffered channels provide limited slack — not infinite queues.

Fan-out / fan-in

Split work across workers (fan-out), merge results (fan-in) with a WaitGroup + results channel.

Failure modes to watch while coding

- Unbuffered infinite job pileup in memory via huge buffers
- No context cancellation on shutdown
- Result channel deadlock when workers exit wrong

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- concurrent.futures.Executor ≈ worker pool.
- Unbounded thread spawns ≈ unbounded go statements — both bad.
- Queue maxsize ≈ buffered channel capacity.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Tune worker count with benchmarks and external limits (DB pools, CPU).

Practice checklist

Before the exercise, confirm you can:

1. Explain “Worker Pools, Fan-in/out, Backpressure” aloud in under two minutes without notes.

2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Unbuffered infinite job pileup in memory via huge buffers
- No context cancellation on shutdown
- Result channel deadlock when workers exit wrong

Exercises

Work in `exercises/32-worker-pools/`. Read that folder's `README.md` before you code.

What you will build

Implement pool semantics: workers process all jobs; results complete.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/32-worker-pools/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why bound workers?
2. What is backpressure?
3. How merge results safely?

Further reading

- https://go.dev/blog/pipelines